

Lập trình hướng đối tượng

Các phương pháp implement cơ bản của OOP

- Định nghĩa class
- Sinh object
- Gọi method

【Java】

Person

Thuộc tính :

lastName:String
firstName:String
age:int

Thao tác :

Person(l:String, f:String, a:int)
getFullName():String
setLastName(l:String):void
printInfo():void

Class name

Tên thuộc tính : kiểu data

Tên thao tác (đối số) : Giá trị trả về

Định nghĩa class

■ Định nghĩa class

● Cú pháp cơ bản

```
modifier class class name {
```

```
// Định nghĩa thuộc tính (field)
```

```
// Định nghĩa thao tác (method)
```

Gọi chung gọi là 「member」 của class

```
}
```

Class name thông thường sẽ sử dụng
chữ tiếng Anh in hoa ở đầu

● Ví dụ

```
public class Person {
```

```
// Định nghĩa field và method
```

```
}
```

Định nghĩa class

- Ở Java, sẽ miêu tả program theo đơn vị class.
 - Trong class, sẽ định nghĩa thuộc tính và thao tác.
 - field và method định nghĩa trong class được gọi chung là 「member」 của class.
 - 「thuộc tính」được miêu tả như là 「biến số」. Biến số thể hiện thuộc tính người ta còn gọi là field.
 - 「thao tác」được miêu tả như là 「method」.

modifier	Nội dung trong nó
public	Có thể access từ bất kỳ đâu
(không chỉ định)	Chỉ có thể access từ class trong cùng package

Định nghĩa thuộc tính

■ Định nghĩa thuộc tính (biến instance)

● Cú pháp cơ bản

Nếu lược giá trị khởi tạo thì sẽ trở thành giá trị default của kiểu data.

modifier kiểu data tên biến;

modifier kiểu data tên biến = giá trị khởi tạo;

● Ví dụ

Thông thường, modifier của biến instance sẽ là private (không công khai ra bên ngoài)

```
public class Person {  
    // Định nghĩa thuộc tính
```

```
    private String lastName;    // Họ  
    private String firstName;  // Tên  
    private int age;           // Tuổi
```

```
    // Định nghĩa method
```

```
}
```

Nếu lược giá trị khởi tạo đi thì

- Trường hợp là kiểu String: null
- Trường hợp là kiểu int: 0

Các loại biến

- **Biến local**

- Biến local là biến được định nghĩa bên trong method, sau khi được định nghĩa nó sẽ trở thành biến hợp lệ bên trong block ({ }) đã định nghĩa.
- Mỗi khi method được gọi ra thì một khu vực sẽ bị chiếm chỗ, và giá trị của biến chỉ được duy trì trong khoảng thời gian method được thực hiện.
- Đây là biến mà người tạo program sử dụng để làm việc. Nó được sử dụng trong các trường hợp bảo quản data tạm thời.
- Biến local không phải là phần tử của mảng thì sẽ không được khởi tạo. Hãy gán giá trị phù hợp để sử dụng.
- Biến mà được khai báo và sử dụng bên trong main method được gọi là biến local.

Các loại biến

- Biến instance
 - Biến thể hiện thuộc tính mà các object đang có thì gọi là biến instance.
 - Giá trị của biến được duy trì cho đến khi object bị phá hủy.
 - Biến instance được định nghĩa bên trong định nghĩa class và bên ngoài định nghĩa method.
 - Cú pháp hầu như giống với cú pháp của biến local được định nghĩa ở trong method nhưng nội định nghĩa, có thể chỉ định modifier và giá trị khi lược giá trị khởi tạo lại khác nhau.
- Biến static: là biến được xác định theo đơn vị class.

Modifier

Access modifier	Explanation
public	Có thể access từ bất kỳ đâu
protected	Chỉ có thể access từ class hoặc sub class có quan hệ kế thừa trong package.
(Không chỉ định)	Chỉ có thể access từ class trong cùng package.
private	Chỉ có thể access từ trong cùng class.

Trường hợp là thuộc tính thì private là phổ biến

Non-Access modifier	Explanation
static	Thuộc về class
final	Không được thay đổi/ghi đè
abstract	Chưa hoàn thiện, cần triển khai
synchronized	Đồng bộ thread
native	Cài đặt bằng ngôn ngữ khác
transient	Bỏ qua khi serialization
...	...

Định nghĩa method (1)

■ Định nghĩa method không có giá trị trả về

● Cú pháp cơ bản

Trường hợp không có giá trị trả về thì sẽ chỉ định
「void」 cho 「kiểu giá trị trả về」

```
modifier void tên method(kiểu data đối số1, ... ) {
    // Xử lý
}
```

Chỉ định trong trường hợp nhận giá trị đã chỉ định
khi gọi method.

● Ví dụ

```
public class Person {
    // Định nghĩa field (thuộc tính)
    private String lastName;
    // Định nghĩa method
```

Tên method thông thường sẽ dùng kí
tự alphabet thường ở đầu

```
public void setLastName(String lname) {
    lastName = lname;
}
```

Set giá trị (lname) đã nhận bằng
đối số vào thuộc tính (lastName)
thể hiện họ

modifier sẽ sử dụng trong method thông thường sẽ
là public (công khai ra ngoài)

* Code giữa chừng đã bị lược đi.

Câu lệnh return

- Câu lệnh return dùng khi kết thúc xử lý trong method mà muốn quay trở lại gốc gọi ra của method.
- Trường hợp định nghĩa method có giá trị trả về, thì nhất thiết phải sử dụng câu lệnh return để trả về giá trị.
- Trường hợp method không có giá trị trả về (chỉ định void) thì không cần câu lệnh return.

return biểu thức hoặc giá trị;

Quay về gốc gọi ra bằng câu lệnh return
Kết quả xử lý không được trả về.

```
public void method1() {  
    if (hantei == true) {  
        // xử lý  
        return;  
    }  
    // xử lý  
}
```

Trường hợp không có câu lệnh return, thì sẽ tiến đến cuối method, quay trở về gốc gọi ra, kết quả xử lý không được trả về.

```
public void method2() {  
    if (hantei == true) {  
        // xử lý  
    }  
    // xử lý  
}
```

Quay về gốc gọi ra bằng câu lệnh return
Kết quả xử lý được trả về

```
public string method3()  
{  
    // xử lý  
    return str;  
}
```

Constructor

■ Constructor

Được gọi ra tự động khi sinh object.

Là hàm định nghĩa xử lý sẽ tiến hành lúc sinh object.

● Cú pháp cơ bản

Nhất thiết phải chỉ định tên class trong tên của constructor

```
modifier    tên constructor( kiểu data đối số 1, ... ) {
    // xử lý ban đầu khi sinh object
}
```

Có thể nhận giá trị đã chỉ định khi sinh object.

● Ví dụ (Khi sinh object, sẽ set giá trị đã chỉ định ở đối số vào biến instance.)

```
public class Person {
```

Tên constructor định nghĩa ở trong Person class là Person

```
    public Person (String lname, String fname, int a) {
        lastName = lname;
        firstName = fname;
        age = a;
    }
}
```

cú pháp giống với method nhưng trong constructor không có chỉ định giá trị trả về

※Code giữ a chừng đã lược

Constructor

- Khi định nghĩa constructor sẽ có thể mô tả xử lý tiến hành khi sinh object.
- Vì có thể chỉ định đối số như method nên có thể nhận giá trị sẽ set cho thuộc tính.
- Có thể định nghĩa nhiều constructor có số lượng và kiểu data của đối số khác nhau.
- Default constructor
 - Trường hợp trong class không định nghĩa 1 constructor nào(trường hợp 0 cái) thì default contract (không có đối số, phần body rỗng) sẽ được tự động add thêm vào lúc compile.

Sinh/giải phóng object (2)

■ Giải phóng object

● Giá trị null

Giá trị thể hiện rằng không có bất kỳ tham chiếu của object nào.

● Cú pháp cơ bản

Biến có tham chiếu của object = null;

● Ví dụ

```
Person personA;  
personA = new Person();
```

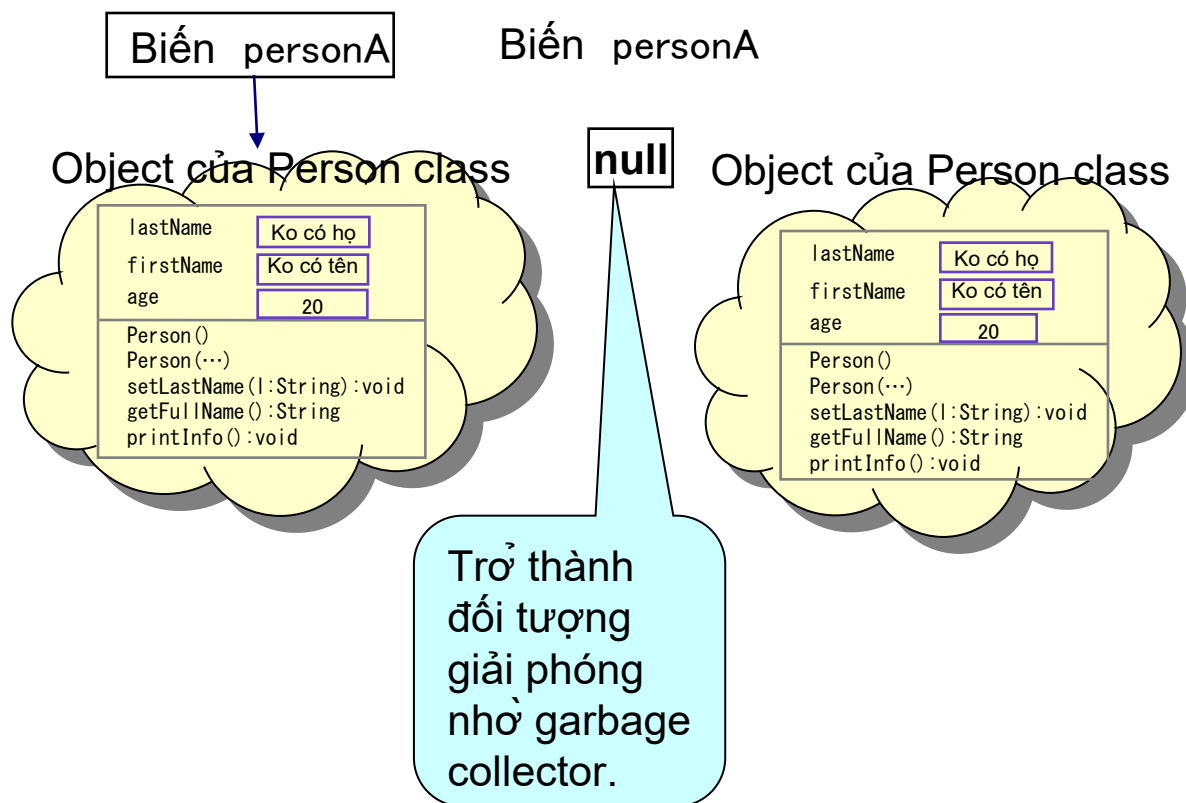
Giá trị default của biến coi class là kiểu data cũng là giá trị null.

```
// Gọi ra method đối với personA. v. ...
```

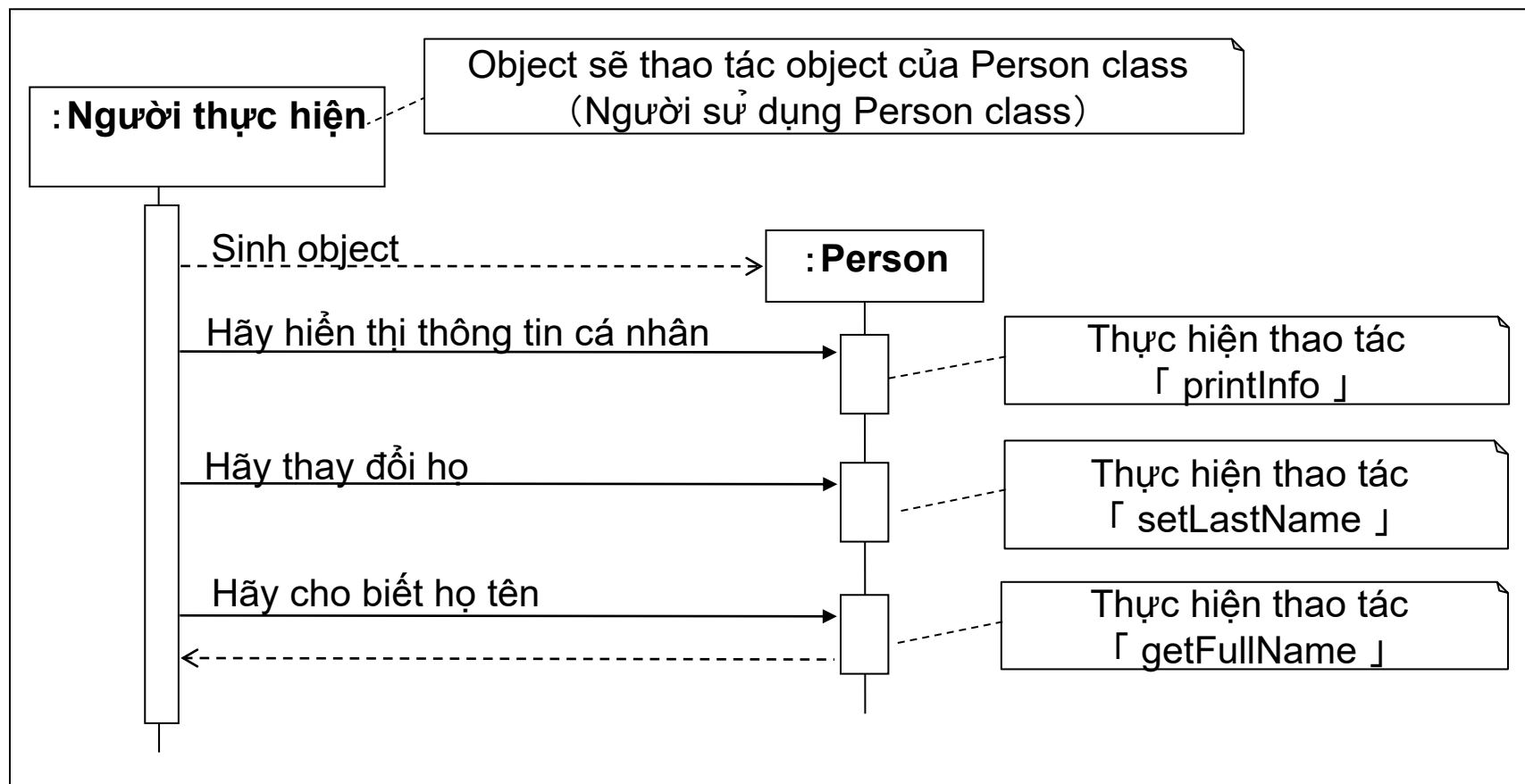
```
personA = null;
```

Giải phóng object

- Nếu như object trở nên không cần thiết nữa thì gán giá trị null vào biến dùng để thao tác object.
- Nếu gán giá trị null thì không có nghĩa, là vùng nhớ mà object đó đang sử dụng sẽ được giải phóng ngay lập tức nhưng chức năng garbage collector sẽ giúp tiến hành giải phóng vùng nhớ nào đó mà object không được tham chiếu từ bất kỳ đâu đang sử dụng.
- Ví dụ:



Gọi ra method



Gọi ra method

■ Gọi ra instance method

● Cú pháp cơ bản

List đối số sẽ chỉ định những đối số tương ứng với định nghĩa method

Tên biến có tham chiếu của object . Tên method(List đối số)

● Ví dụ

Kể cả trường hợp không có việc truyền nhận đối số thì () cũng cần thiết

Chú ý !

```
Person personA = new Person("佐藤 (Sato)", "花子 (Hanako)", 22);
```

```
// Hiển thị thông tin cá nhân
```

```
personA.printInfo(); // Ví dụ gọi ra method không có đối số, không có giá trị trả về
```

```
// Thay đổi họ
```

```
personA.setLastName("川村 (Kawamura)"); // Ví dụ gọi ra method có đối số, không có giá trị trả về.
```

```
// Get họ tên
```

```
String fullName = personA.getFullName();
```

```
// Ví dụ gọi ra method không có đối số, có giá trị trả về
```

```
System.out.println("変更後の姓名は(Họ tên sau khi thay đổi là)" + fullName + "です");
```

Overload

■ Overload

Là việc định nghĩa các method hoặc constructor cùng tên nhưng có đối số khác nhau trong cùng class.

Đối số giống nhau nghĩa là 「số lượng của đối số」 「kiểu data」 「trình tự chỉ định của đối số」 là hoàn toàn giống nhau.

● Ví dụ tốt

```
void proc1() { ... }
void proc1(int i, String str) { ... }
void proc1(String str, int i) { ... }
```

List đối số (int , String) và (String , int) là những method(constructor) riêng biệt

● Ví dụ xấu

```
void proc2(int x) { ... }
boolean proc2(int x) { ... }
```

Nếu định nghĩa method có đối số giống nhau, giá trị trả về khác nhau sẽ bị compile error.

```
void proc3(int x) { ... }
void proc3(int y) { ... }
```

Chú ý !

Do dù tên biến của đối số có khác nhau đi nữa nhưng nếu kiểu data giống nhau thì vẫn bị compile error.

Biến static, static method (1)

■ Biến static

Là biến có vùng nhớ được đảm bảo theo đơn vị class và có thể sử dụng được ngay cả khi không có object.

Sử dụng static keyword để khai báo

Trường hợp access vào biến static thì sẽ chỉ định là「class name.tên biến」

● Ví dụ

【Biến static được định nghĩa trong Person class】

```
public static final String SPECIES = "Homo sapiens";
```

Có nhiều trường hợp để modifier của biến static là public (công khai ra bên ngoài) , final (hằng số)

【Ví dụ sử dụng】

```
System.out.println("種(chủng loại) : " + Person.SPECIES);
```

Ngay cả khi không tồn tại object của class có biến static thì vẫn có thể sử dụng được.

Biến static, static method (2)

■ static method

Có thể sử dụng cho dù không có object

Sử dụng static keyword để khai báo

Trường hợp gọi ra method static thì chỉ định 「class name.tên method (list đối số)」

● Ví dụ

【static method đang được định nghĩa trong Employee class】

```
public static final String SPECIES = "Homo sapiens";  
public static void printSpecies() {  
    System.out.println(SPECIES); // Chỉ định SPECIES thành Person.SPECIES  
    cũng được  
}
```

【Ví dụ sử dụng】

modifier sử dụng ở static method thông thường là public (công khai ra bên ngoài)

```
Person.printSpecies();
```

Có thể sử dụng ngay cả khi không tồn tại object của class có static method.

MAIN METHOD

■ main method

● Cú pháp cơ bản

Là method được gọi ra đầu tiên khi thực hiện application.

```
public static void main( String[] args ) {
    // Xử lý
}
```

Được truyền như là mảng String.

args[0] ← giá trị 1
args[1] ← giá trị 2
:

● Đối số của main method (nhận giá trị từ command)

```
java class name giá trị 1 giá trị 2 ...
```

※ Chỉ định các giá trị muốn truyền cho main method ngăn cách nhau bằng blank vào sau class name

※ Do đã được chỉ định là public nên có thể gọi ra từ bên ngoài.

※ Do đã được chỉ định là static nên có thể gọi ra cho dù không tồn tại object.

※ Do không có giá trị trả về nên chỉ định 「void」 vào phần 「kiểu giá trị trả về」.

※ Tên method là 「main」.

※ main method có thể nhận giá trị đã được chỉ định khi thực hiện như là một mảng String.

「args」 là tên biến nên có thể chỉ định tên tùy ý.